JTS Topology Suite

TestRunner & TestBuilder User Guide

Rebuilt from the 3 February 2005 text

Base text: 3 February 2005 · rebuild: 16 August 2026

This guide describes how to use the JTS TestRunner, a Java application that validates the JTS Topology Suite, and the JTS TestBuilder, a Java application that creates, visualizes, and runs geometry tests. The 2005 chapter plan is kept. Startup, packages, and curve drawing are brought up to this tree.

Package names in this tree are `org.locationtech.jts`. The 2005 copies used `com.vividsolutions.jts`. LocationTech JTS remains the upstream project; this fork is not described as if it were already upstream.

Document change control

Revision	Date	Author	Change
1.0	13 Sep 2001	Jonathan Aquino	Original draft.
1.1	17 Dec 2001	Jonathan Aquino	Added TestBuilder documentation.
1.2	30 Jan 2002	Jonathan Aquino	Added ACME licence (historical).
1.2-rebuild	16 Aug 2026	This tree	LaTeX rebuild. Startup, packages, curve drawing, WKT apply, A/B colours, OverlayNGCurve, Hausdorff. ACME appendix replaced with the JTS dual licence.

Contents

1 Overview

The TestRunner and TestBuilder are used by people who are developing or evaluating JTS. While similarly named, the JTS TestRunner is not the JUnit TestRunner. JUnit is a general Java testing framework. The JTS TestRunner is easier for testing JTS computations because tests are specified in XML text files — no Java coding or compilation. Those files can be created by hand or generated by the TestBuilder.

The 2005 guide required Java 2 JRE/SDK v1.3. This tree targets Java 8 and above. Download a current JDK from the vendor of your choice; the 2005 `java.sun.com/products` link is historical.

The 2005 class names were `com.vividsolutions.jtstest.testrunner.TopologyTestApp` and `com.vividsolutions.jtstest.testbuilder.JTSTest`. On this tree they are `org.locationtech.jtstest.testr` and `org.locationtech.jtstest.testbuilder.JTSTestBuilder`. The usual way to start TestBuilder is the built jar, not a hand-built classpath of `jdom.jar` / `xerces.jar` / `jts.jar` / `jts_test.jar`.

This guide is divided into two main sections: Using the TestRunner, and Using the TestBuilder. An appendix details the XML test-file format. For more information on JTS, see the JTS Technical Specifications.

2 Using the TestRunner

This section describes how to use the TestRunner to run a test file that validates the JTS Topology Suite. Test files are created using the TestBuilder (Section 3) or by hand (Section 5).

2.1 Starting

From a built tree, run the TestRunner via the scripts in `bin/` or:

```
java -cp modules/tests/target/jts-tests-*-SNAPSHOT.jar \
     org.locationtech.jtstest.testrunner.JTSTestRunnerCmd \
     -files path/to/tests.xml
```

The 2005 GUI mode (`-gui` / `testrunner.bat`) and the switches `-files`, `-properties`, `-verbose` are the same idea: a list of XML files, a results panel or text listing, and a status line of cases / tests / passed / failed / exceptions.

Graphical mode has file-chooser dialogs that make it easy to build a list of test files. Text mode is quicker to start. The `-files` switch can be used to run all test files in a given directory.

2.2 Reading the results

A case specifies two geometries (A and B). A test specifies an operation and its expected value. The status line reports counts of cases, tests, passes, failures, thrown exceptions, and parse exceptions.

Statistic	Meaning
Cases	Each case specifies two geometries to test.
Tests	Each test specifies an operation on those geometries.
Passed	Actual value matches expected value.
Failed	Actual value differs from expected value.
Exception	An error occurred inside JTS or the TestRunner.
Parse exception	The test file has a syntax error.

Passed-test detail is shown only with `-verbose`. A failed test means either fix the test or fix JTS. A parse exception is a problem in the XML or WKT (a missing comma in a LINESTRING is the 2005 example and is still the usual cause).

2.3 Switches

Switch	Description
`-files`	List of test files and directories. Separate items with spaces.
`-properties`	A `.properties` file that remembers the last file list.
`-gui`	Graphical mode, when the build still ships that entry point.
`-verbose`	Show passed tests as well as failures.

The easiest way to specify every file in a directory is to pass the directory to `-files`. Any combination of switches is permitted. If no switches are specified, a description of the switches is displayed.

> **Amendment.** XML `<a>` / `<b>` bodies that contain `CIRCULARSTRING` / `COMPOUNDCURVE` / `CURVEPOLYGON` / `MULTICURVE` / `MULTISURFACE` must be parsed with a curve-capable reader. On this tree the TestRunner / TestCase path uses `CurveWKTReader`.
>
> A test that calls `Geometry.intersection` on a curve is not `OverlayNGCurve`. Name the OverlayNGCurve function if that is what you mean to assert.
>
> A `DiscreteHausdorffDistance` assertion is the public API: two certified pairs are closed-form; everything else is chords. `fail()` stays on the general case.

3 Using the TestBuilder

The 2005 guide called TestBuilder experimental, with a useful starting point for creating, visualizing, and running test files. That description is still fair. The application on this tree is the current LocationTech JTS TestBuilder plus the curve tools that are actually merged.

Useful features:

- creating test geometries visually or by entering Well-Known Text;
- snapping to a grid;
- specifying expected values for spatial functions and the intersection matrix;

- opening and saving XML test files.

The 2005 "Save As HTML" bulk documentation generator is not claimed here as a supported workflow. The `vividsolutions.com/jts/tests` URL in the 2005 copy is historical.

3.1 Starting the TestBuilder

Build the app module, then from the project root:

```
java -jar modules/app/target/JTSTestBuilder.jar -Xmx2000M
```

On macOS, if the native look-and-feel misbehaves:

```
java -Dswing.defaultlaf=javax.swing.plaf.metal.MetalLookAndFeel \
     -jar modules/app/target/JTSTestBuilder.jar -Xmx2000M
```

Do not start it as the 2005 `java com.vividsolutions.jtstest.testbuilder.JTSTest` with a four-jar classpath. That layout is gone.

3.2 Opening a test file

File / Open XML File(s). The TestBuilder displays one test case at a time: a pair of geometries and their tests. Use Previous / Next or the Tests list to move. Pan, zoom, zoom 1:1, and zoom-to-extent examine the geometries. The WKT tab shows Well-Known Text.

3.3 Precision model

A test file's precision model is set from the TestBuilder precision-model dialog (Edit / Precision Model). The default is Floating. See the Technical Specifications.

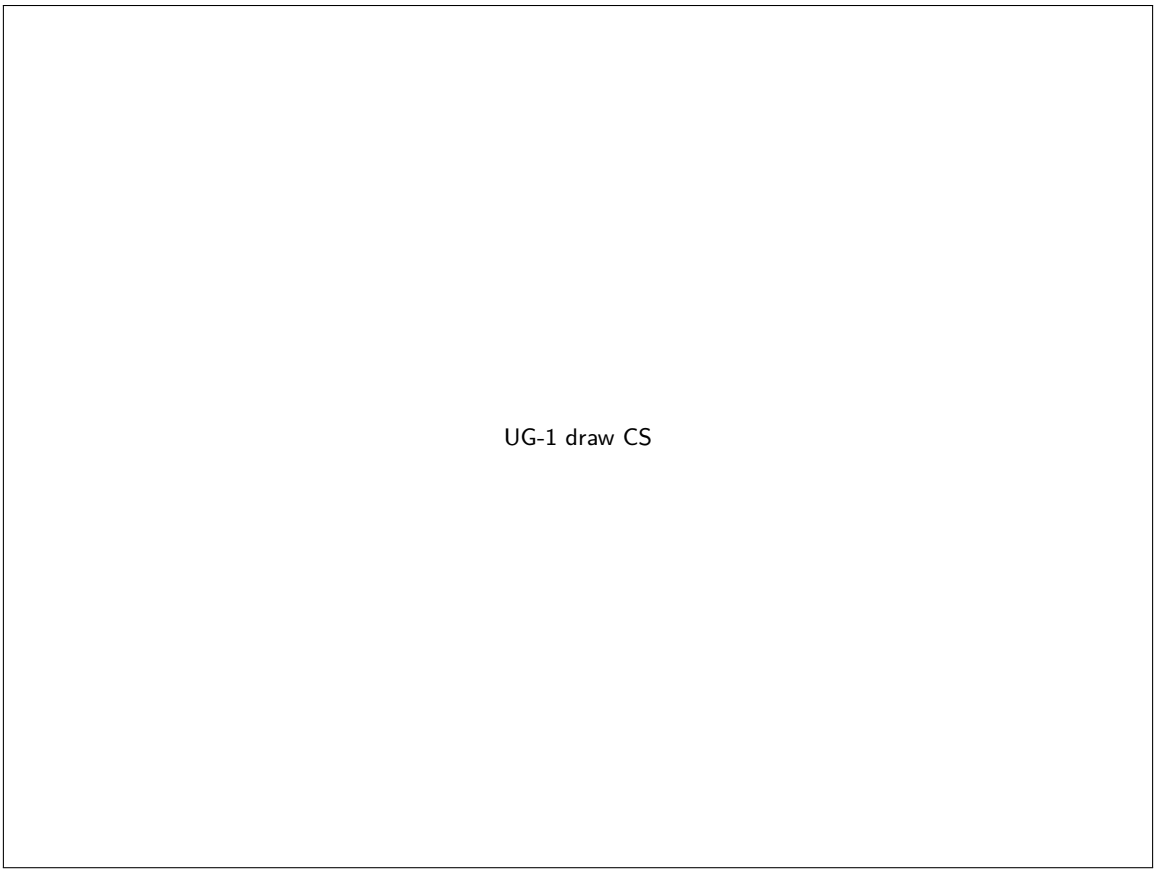
3.4 Creating geometries

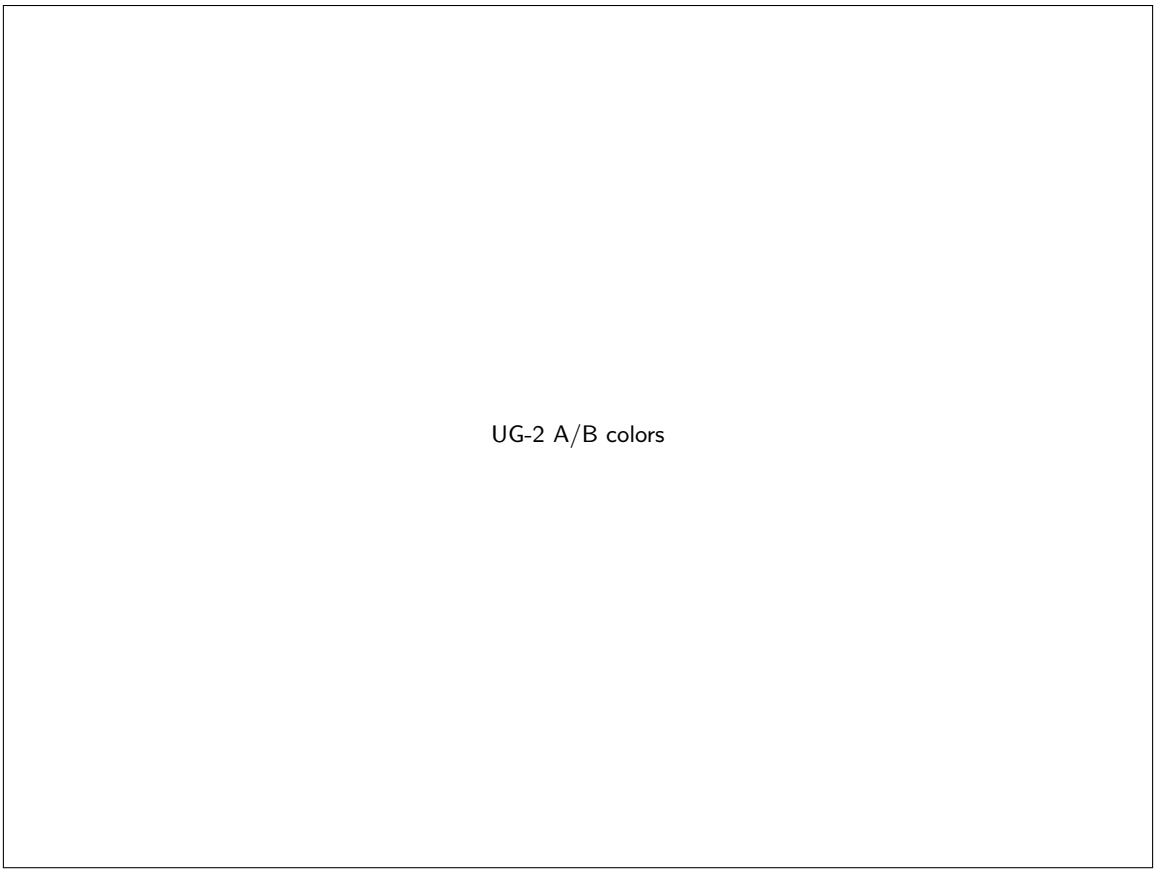
The 2005 workflow is still the right picture: pick geometry A or B, pick a draw tool, click in the view. Geometry A is shown in blue; geometry B, in red. You can also type WKT. Erase, undo point, and undo part still apply.

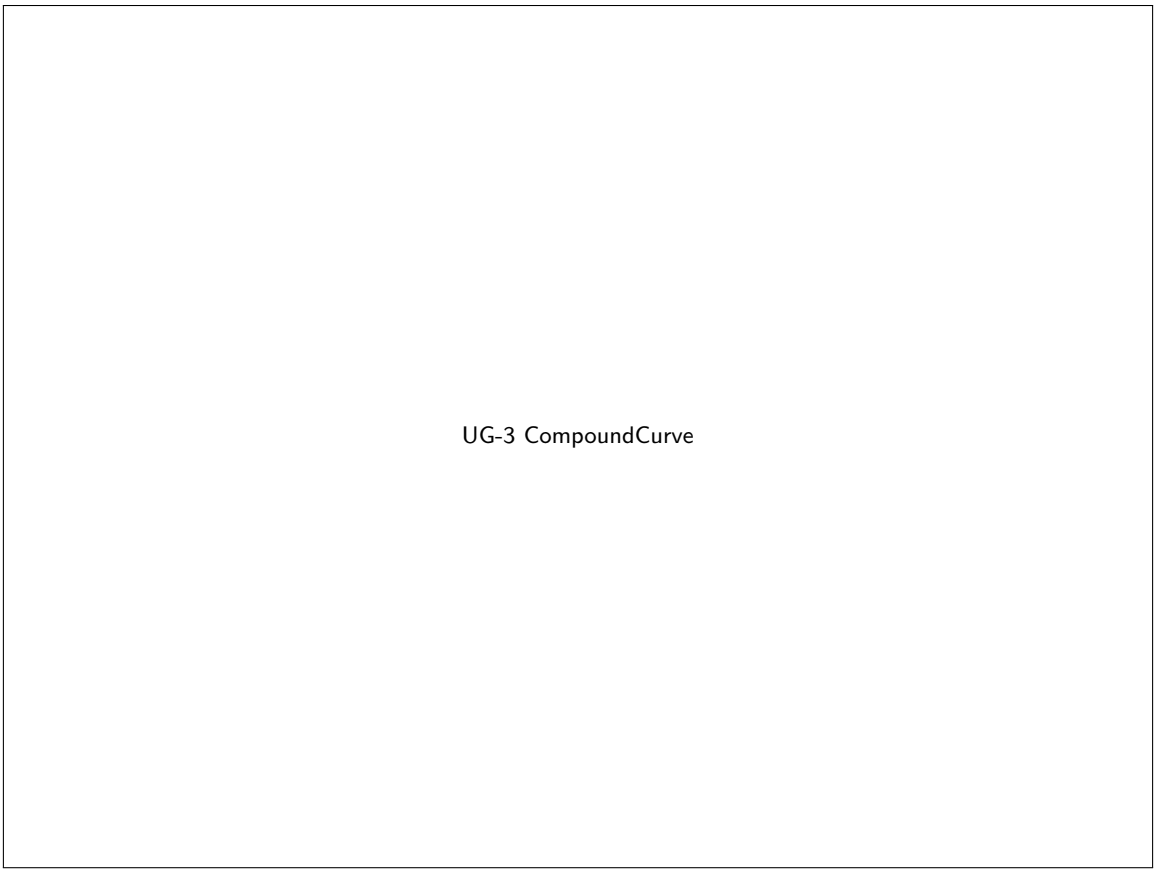
On this tree the draw tools include the linear set (point, linestring, polygon, rectangle) and the curve set that is actually merged: `CircularString`, `CompoundCurve`, `CurvePolygon`. Triangle and TIN tools also exist.

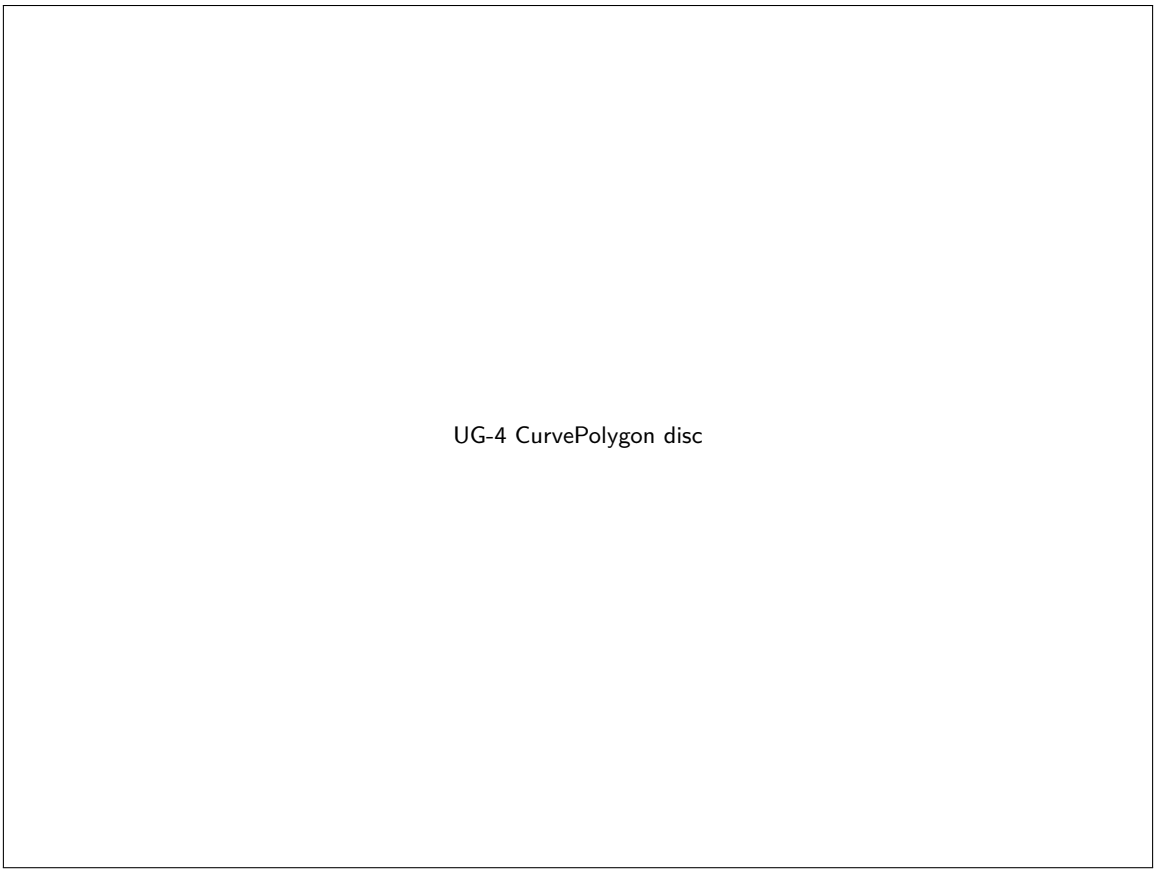
Draw-tool icons follow the A/B focus: index 0 (A) is blue-dominant; otherwise (B) is red-dominant. That applies to the linear draw tools and to CircularString / CompoundCurve / CurvePolygon.

WKT apply uses `CurveWKTReader`, so `CIRCULARSTRING` pasted into the WKT tab is a `CircularString`, not a parse error. Enter in the A or B WKT pane loads that pane. The Clear control is labeled and does not wipe the other pane on apply.


UG-1 draw CS


UG-2 A/B colors


UG-3 CompoundCurve



A clothoid may be added as a non-leading `COMPOUNDCURVE` member from the CompoundCurve tool and from the clothoid playground extras. That path is extras, not a closed-form construction.

3.5 Predicates and functions

The 2005 Predicates tab (expected DE-9IM, binary predicates) and Functions tab (union, intersection, buffer, ...) are the same idea. The current UI is a function tree: pick a function, run it, inspect the result geometry and its WKT.

`OverlayNGCurve` is registered as its own function class (intersection / union / difference / symDifference). That is the curve overlay surface. `Geometry.intersection` in the Geometry function group is not OverlayNGCurve.

Closed-form constructions exposed as functions include disc and single-arc convex hull, the compound-curve hull of circular-plus-straight members, `MaximumInscribedCircle` on a certified disc and on a stadium, and `LargestEmptyCircle` on legal curve obstacles.

3.6 Saving

File / Save As XML writes the current cases. Default / export WKB paths route through `CurveWKBWriter`, so a saved curve keeps ISO codes 8–12. A bare `WKBWriter.write` call still emits type 2 or 3.

4 Curves in TestBuilder

TestBuilder can draw `CircularString`, `CompoundCurve`, and `CurvePolygon` (toolbar tools plus WKT via `CurveWKTReader`). It also exposes OverlayNGCurve and the distance functions.

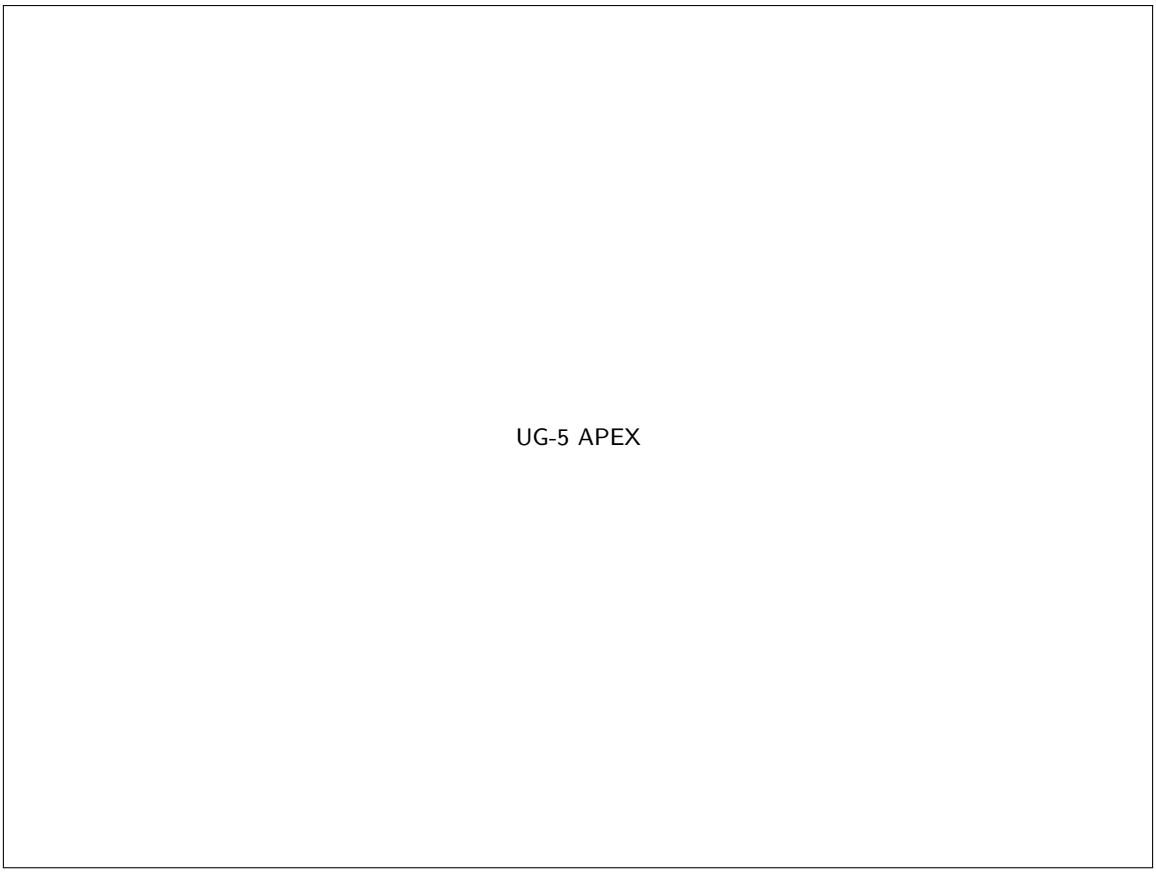
On this tree the concrete SFA/SQL-MM curve types live in the `jts-curve` module (`org.locationtech.jts.geom.cu`
Construction is opt-in: the default `GeometryFactory` throws on the curve `create*` methods. Use `CurveGeometryFactory`. Constructors do not linearise.

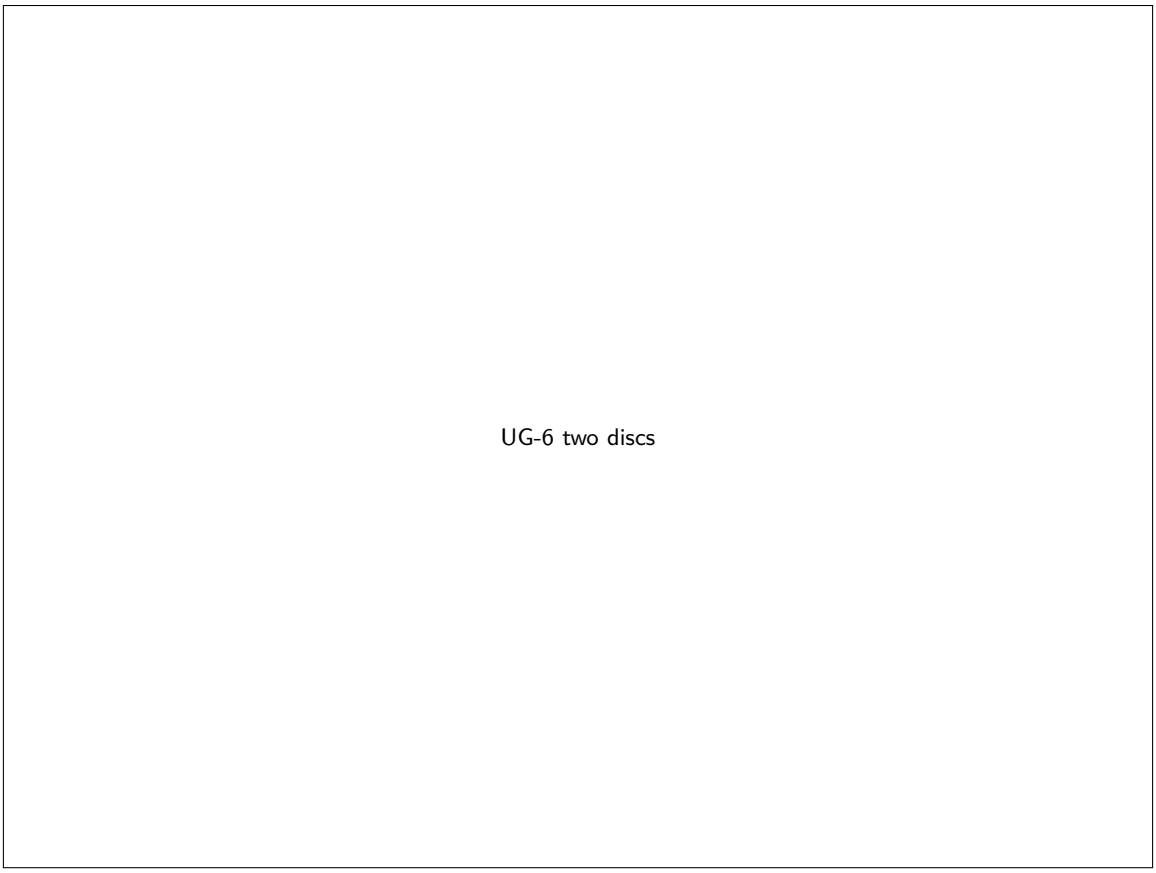
Core `WKTReader` still rejects `CIRCULARSTRING` and the other SQL-MM keywords. Read them with `CurveWKTReader`. Write structured members with `CurveWKTWriter`.

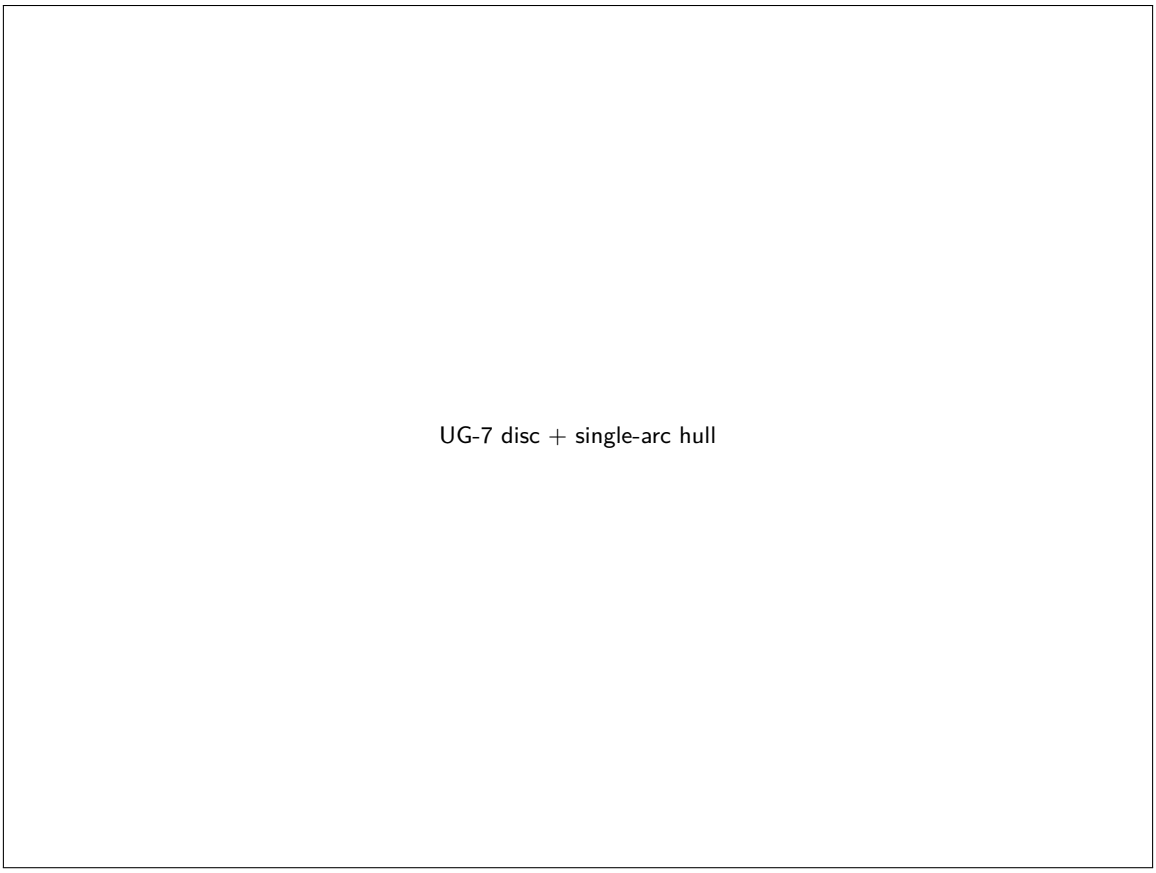
WKB type codes 8–12 are first-class in core `WKBReader`. `CurveWKBWriter` emits codes 8–12. A bare `WKBWriter` still emits type 2 (CircularString / CompoundCurve) or 3 (CurvePolygon).

The curve overlay type is `OverlayNGCurve`. There is no public noder. Closed-form overlay answers are limited to certified discs.

Public `DiscreteHausdorffDistance` (commit `0ca71b`) has a closed form for two pairs only: (1) a single-arc `CircularString` toward a single-segment `LineString`, apex $\sqrt{949}/6 - 7/6 \approx 3.967641$ for `CIRCULARSTRING (0 0, 2 3, 10 0)` vs `LINESTRING (0 0, 10 0)`; (2) two circular discs. A `MultiSurface` unwrap of one disc is the same pair. The exact path skips densify; densify is not the laser. For every other pair the public API still sees vertices / chords. Fréchet remains open.




UG-6 two discs



5 Appendix: test file format

A test file is XML. It contains one or more cases; each case contains one or more tests. A case specifies two geometries. A test specifies an operation and its expected result.

5.1 Example

```
<run>
  <desc>example</desc>
  <precisionModel type="FLOATING"/>
  <case>
    <desc>point in a polygon</desc>
    <a>POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))</a>
    <b>POINT (5 5)</b>
    <test>
       <op name="contains">true</op>
    </test>
  </case>
  <case>
    <desc>single-arc D-HF witness</desc>
    <a>CIRCULARSTRING (0 0, 2 3, 10 0)</a>
    <b>LINESTRING (0 0, 10 0)</b>
    <test>
       <op name="orientedDistance" arg1="a" arg2="b">3.967641</op>
```

```
    </test>
  </case>
</run>
```

The second case is the D-HF closed-form witness on this tree (apex $\sqrt{949}/6 - 7/6$). A case that is not one of the two certified pairs must not be written as if `DiscreteHausdorffDistance` were arc-length Hausdorff. The first case is the 2005 example, unchanged.

5.2 XML shape

The 2005 DTD-style listing is still the right shape: `<run>` contains `<precisionModel>` and one or more `<case>`; each case has `<a>`, optional `<b>`, and `<test>` / `<op>` with a name and expected value. WKT inside `<a>` / `<b>` may now be a curve keyword; the reader on this tree is `CurveWKTReader`.

6 Appendix: licence

JTS is dual-licensed under the Eclipse Public License 2.0 and the Eclipse Distribution License 1.0. See `LICENSE_EPLv2.txt`, `LICENSE_EDLv1.txt`, and `LICENSES.md`. The 2005 "Appendix: ACME Licence" is not reproduced. It was never the JTS project licence on this tree.

End of TestRunner & TestBuilder User Guide. Named figure holes are empty 4:3 slots (target 1600×1200, full TestBuilder window). 2005 Amyuni screen-shots are omitted; the procedures they illustrated are kept in prose.